

Experiment-6

Aim:

- A. A fruit distribution company records the number of apples and oranges sold each day. For every date, exactly one record exists for apples and one record exists for oranges. Generate a daily sales comparison report showing the difference between the number of apples sold and the number of oranges sold on the same day. The difference should be calculated as: $\text{Difference} = \text{Apples Sold} - \text{Oranges Sold}$. Display the sale date and the calculated difference. The result should be sorted by sale date in ascending order.
- B. A company stores hierarchical employee reporting information in a table. Each record represents a node in a hierarchy where:
- id represents the current node.
 - p_id represents the parent node.
 - Classify every node into one of the following categories:
 - Root: A node that does not have any parent.
 - Leaf: A node that has a parent but does not have any children.
 - Inner: A node that has both a parent and at least one child.
 - Display the node ID along with its category. Return the result sorted by node ID.

Tools / Platform Used:

- Windows
- PostgreSQL
- PgAdmin

Theory:

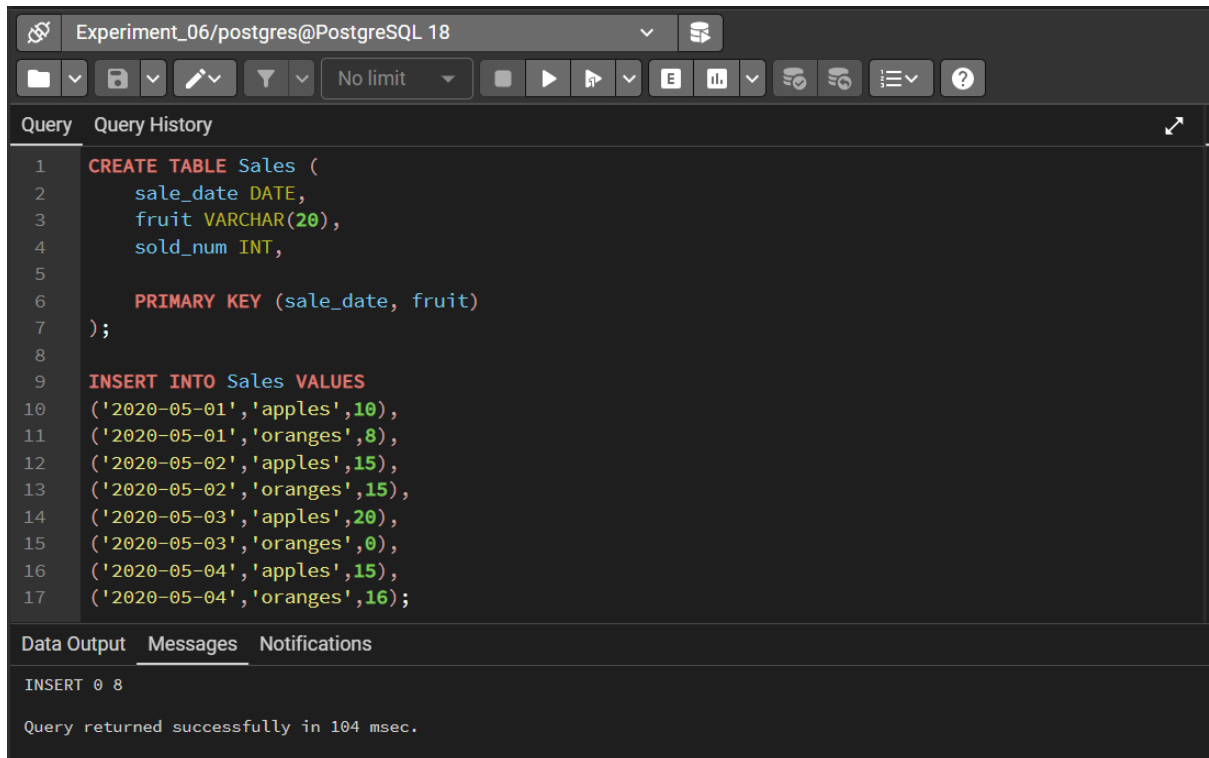
The CASE expression is a conditional expression used to return different values depending on one or more conditions. It is commonly used in:

- SELECT statements
- UPDATE statements
- ORDER BY
- WHERE clauses

It is similar to an if-else-if ladder or a switch-case statement in programming languages.

Code and Output:

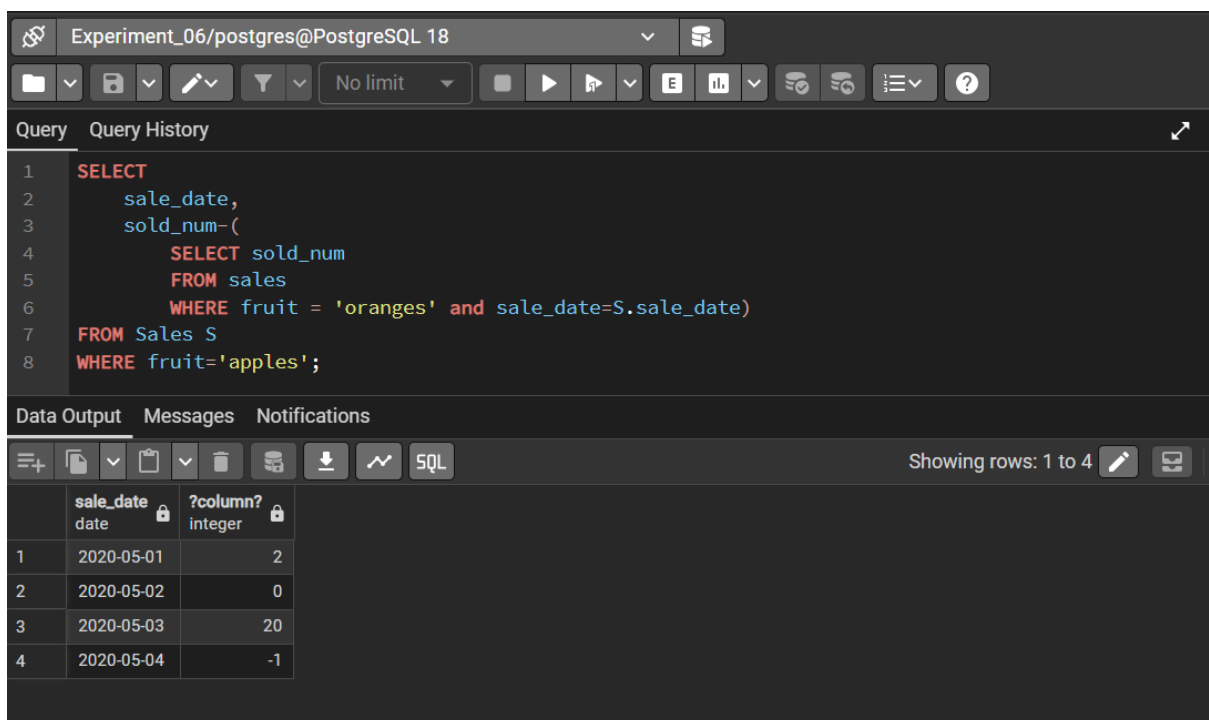
A.



The screenshot shows a PostgreSQL IDE window titled "Experiment_06/postgres@PostgreSQL 18". The query editor contains the following SQL code:

```
1 CREATE TABLE Sales (  
2     sale_date DATE,  
3     fruit VARCHAR(20),  
4     sold_num INT,  
5  
6     PRIMARY KEY (sale_date, fruit)  
7 );  
8  
9 INSERT INTO Sales VALUES  
10 ('2020-05-01','apples',10),  
11 ('2020-05-01','oranges',8),  
12 ('2020-05-02','apples',15),  
13 ('2020-05-02','oranges',15),  
14 ('2020-05-03','apples',20),  
15 ('2020-05-03','oranges',0),  
16 ('2020-05-04','apples',15),  
17 ('2020-05-04','oranges',16);
```

The "Data Output" tab is selected, showing the message "INSERT 0 8" and "Query returned successfully in 104 msec."



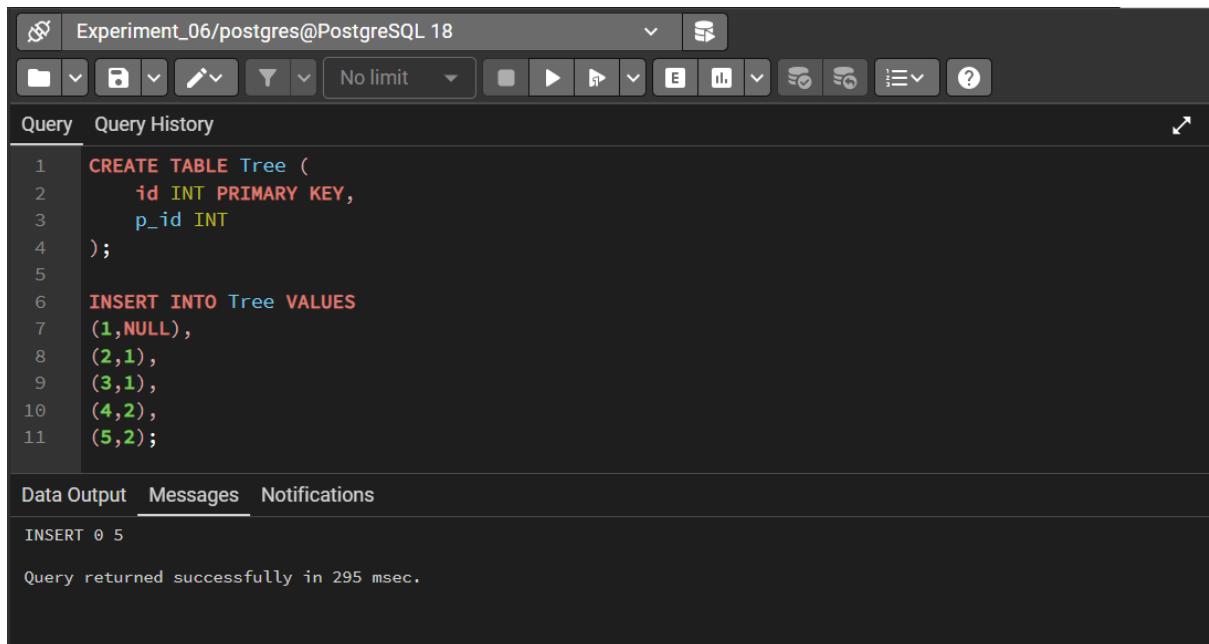
The screenshot shows the same PostgreSQL IDE window. The query editor contains the following SQL code:

```
1 SELECT  
2     sale_date,  
3     sold_num-(  
4         SELECT sold_num  
5         FROM sales  
6         WHERE fruit = 'oranges' and sale_date=S.sale_date)  
7 FROM Sales S  
8 WHERE fruit='apples';
```

The "Data Output" tab is selected, showing a table with 4 rows and 3 columns. The table has columns "sale_date", "sold_num", and "?column?". The data is as follows:

	sale_date	sold_num	?column?
1	2020-05-01	10	2
2	2020-05-02	15	0
3	2020-05-03	20	20
4	2020-05-04	15	-1

B.

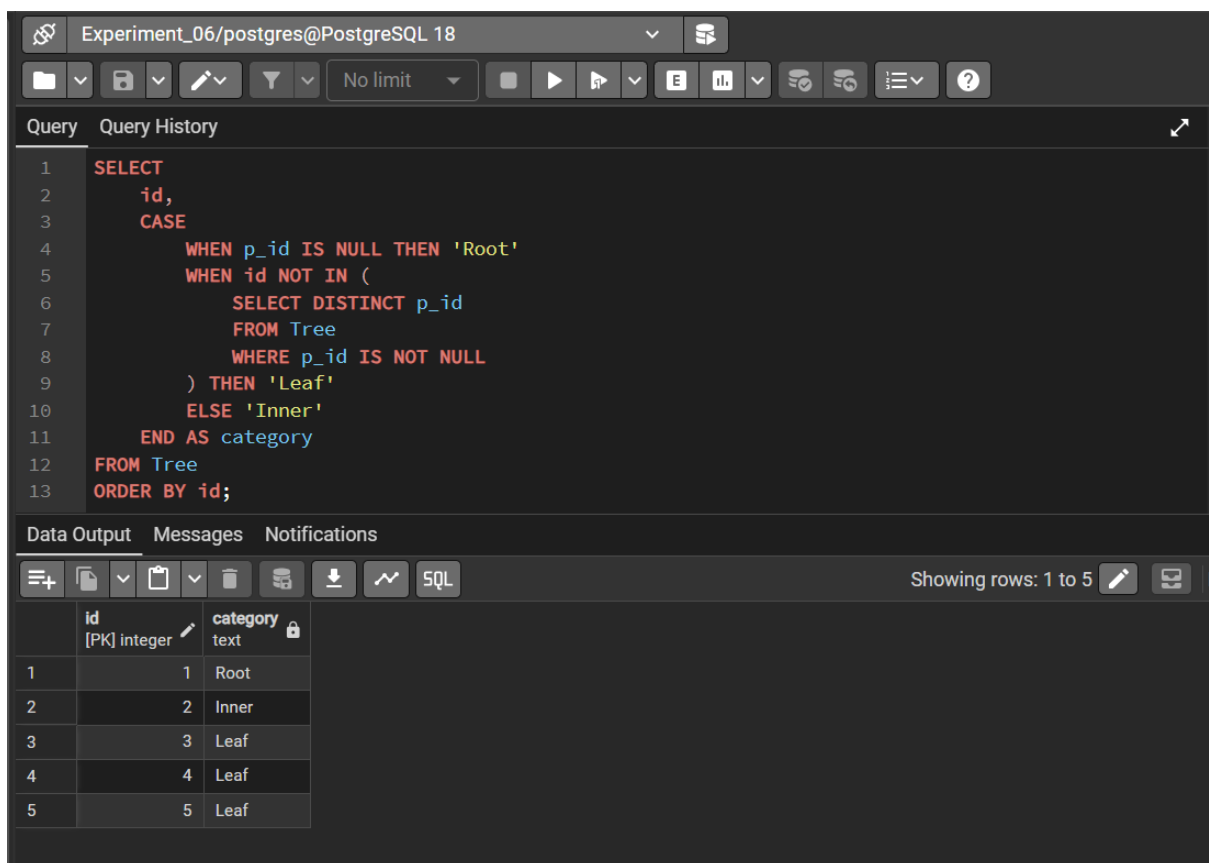


The screenshot shows a PostgreSQL query editor interface. The title bar indicates the connection is 'Experiment_06/postgres@PostgreSQL 18'. The 'Query' tab is active, displaying the following SQL code:

```
1 CREATE TABLE Tree (  
2     id INT PRIMARY KEY,  
3     p_id INT  
4 );  
5  
6 INSERT INTO Tree VALUES  
7 (1,NULL),  
8 (2,1),  
9 (3,1),  
10 (4,2),  
11 (5,2);
```

Below the query editor, the 'Data Output' tab is active, showing the result of the insert statement:

```
INSERT 0 5  
  
Query returned successfully in 295 msec.
```



The screenshot shows the same PostgreSQL query editor interface. The 'Query' tab is active, displaying the following SQL code:

```
1 SELECT  
2     id,  
3     CASE  
4         WHEN p_id IS NULL THEN 'Root'  
5         WHEN id NOT IN (  
6             SELECT DISTINCT p_id  
7             FROM Tree  
8             WHERE p_id IS NOT NULL  
9         ) THEN 'Leaf'  
10        ELSE 'Inner'  
11    END AS category  
12 FROM Tree  
13 ORDER BY id;
```

Below the query editor, the 'Data Output' tab is active, showing the result set. The table has two columns: 'id' (integer, primary key) and 'category' (text). The results are as follows:

id	category
1	Root
2	Inner
3	Leaf
4	Leaf
5	Leaf

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with Switch-Case Statements in PostgreSQL.
- Got hands on experience with integrating subqueries and case statement in PostgreSQL
- Practiced commands of Switch-Case in SQL.